

```
In [57]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score
from qiskit.primitives import Sampler
from qiskit.primitives import StatevectorSampler
from qiskit_aer import Aer
from qiskit_algorithms import VQE
from qiskit.circuit.library import ZZFeatureMap, EfficientSU2
from qiskit_machine_learning.algorithms import VQC
from qiskit_algorithms.optimizers import COBYLA,SPSA
```

```
In [58]: #Step_1: Load Dataset
iris = load_iris()

X = iris.data      # 4 features
y = iris.target    # 3 classes
```

```
In [59]: #Step_2:Normalize Data

scaler = MinMaxScaler(feature_range=(0, np.pi))
X = scaler.fit_transform(X)
```

```
In [60]: # Step_3:Split Data Set for Train and Test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
In [61]: '''#Step_4:
y_train_0 = (y_train == 0).astype(int)
y_train_1 = (y_train == 1).astype(int)
y_train_2 = (y_train == 2).astype(int)'''
```

```
Out[61]: '#Step_4:\ny_train_0 = (y_train == 0).astype(int) \ny_train_1 = (y_train == 1).astype(int)\ny_train_2 = (y_train == 2).as
type(int)'
```

```
In [62]: #Step_5: Create circuit
from qiskit import QuantumCircuit

qc = QuantumCircuit(4)          #|q1q1q2q3>=|0000>
qc.draw('mpl', fold=-1)
```

```
Out[62]:
```

q_0 —

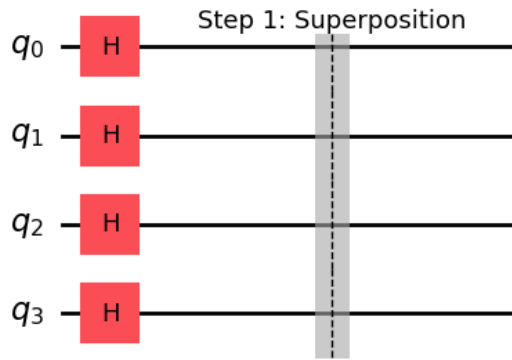
q_1 —

q_2 —

q_3 —

```
In [63]: #Step 6: Create Suoerposition apply Hadamard on all qubits
qc.h(range(4))          # H@4|0000>
qc.barrier(label="Step 1: Superposition")
qc.draw('mpl', fold=-1)
```

Out[63]:



In [64]:

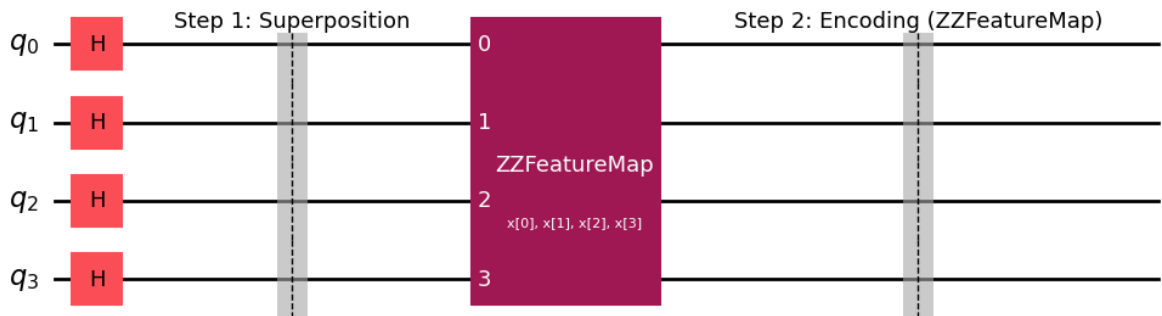
```
#Step7: Feature Encoding from Classical bits to quantum qubits ( Angle encoding) using ZZFeatureMap
from qiskit.circuit.library import ZZFeatureMap

feature_map = ZZFeatureMap(
    feature_dimension=4,
    reps=1,
    entanglement='full'
)

qc.compose(feature_map, inplace=True)
qc.barrier(label="Step 2: Encoding (ZZFeatureMap)")
qc.draw('mpl', fold=-1)
```

#UZZ(x)=e-(iΣxiZi + ΣxixjZiZj)

Out[64]:



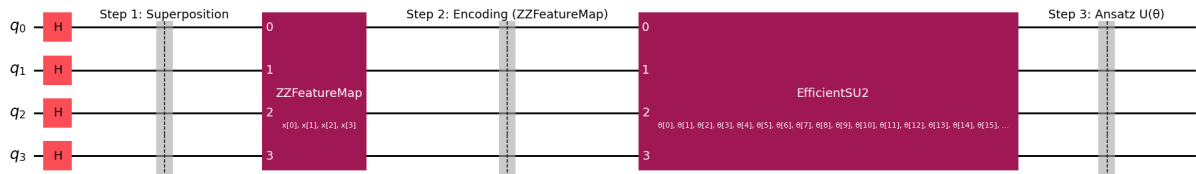
In [65]:

```
#Step 8: Train quantum Circuit using ansatz
from qiskit.circuit.library import EfficientSU2

ansatz = EfficientSU2(
    num_qubits=4,
    reps=3,
    entanglement='full'
)

qc.compose(ansatz, inplace=True)
qc.barrier(label="Step 3: Ansatz U(θ)")
qc.draw('mpl', fold=-1)
```

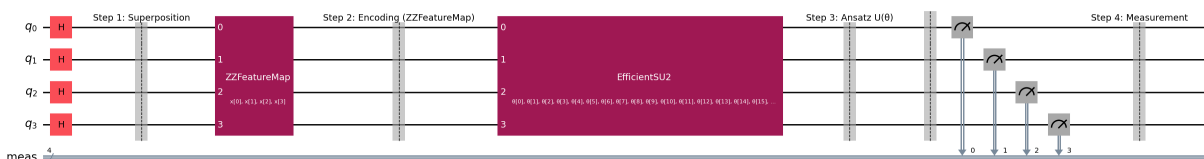
Out[65]:



In [66]:

```
# Step 9: Measure
qc.measure_all()
qc.barrier(label="Step 4: Measurement")
qc.draw('mpl', fold=-1)
```

Out[66]:



In [67]:

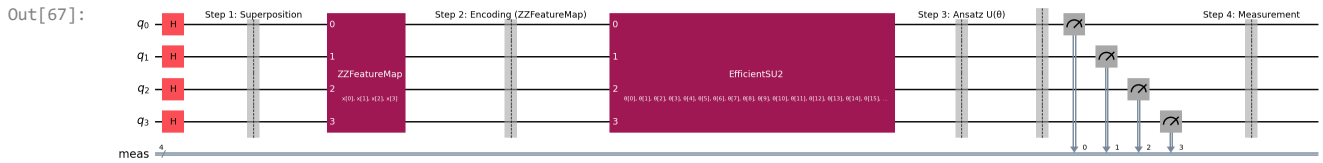
```
#STEP 11: CREATE VQC MODEL
sampler = StatevectorSampler()
```

```

vqc = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=SPSA(maxiter=500),
    #optimizer=COBYLA(maxiter=1000,tol=1e-3),
    sampler=sampler
)
qc.draw('mpl', fold=-1)

```

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.



```

In [68]: #STEP 12: TRAIN MODEL
#vqc.fit(X_train, y_train_0) # example: class 0
vqc.fit(X_train, y_train)

```

Out[68]: <qiskit_machine_learning.algorithms.classifiers.vqc.VQC at 0x2d0cfd02780>

```

In [69]: #STEP 13: PREDICT
y_pred = vqc.predict(X_test)

```

```

In [70]: #STEP 14: ACCURACY
from sklearn.metrics import accuracy_score
print("Accuracy:", accuracy_score(y_test, y_pred))

```

Accuracy: 0.5333333333333333

```

In [71]: from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay

```

```

# Example data
#y_true = [0, 1, 0, 1]
#y_pred = [1, 1, 0, 1]

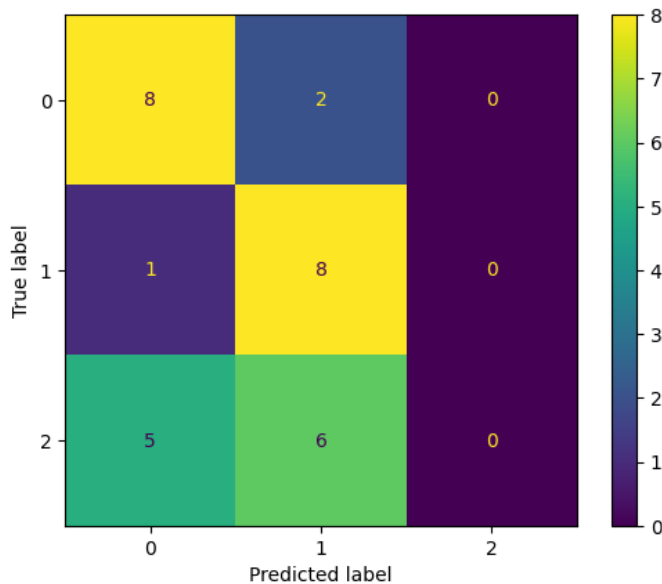
# 1. Get raw matrix
matrix = confusion_matrix(y_test, y_pred)

# 2. Get accuracy score
acc = accuracy_score(y_test, y_pred)

# 3. Plot the matrix
disp = ConfusionMatrixDisplay(confusion_matrix=matrix)
disp.plot()

```

Out[71]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2d0ca6c6690>



```

In [77]: import matplotlib.pyplot as plt

plt.figure()

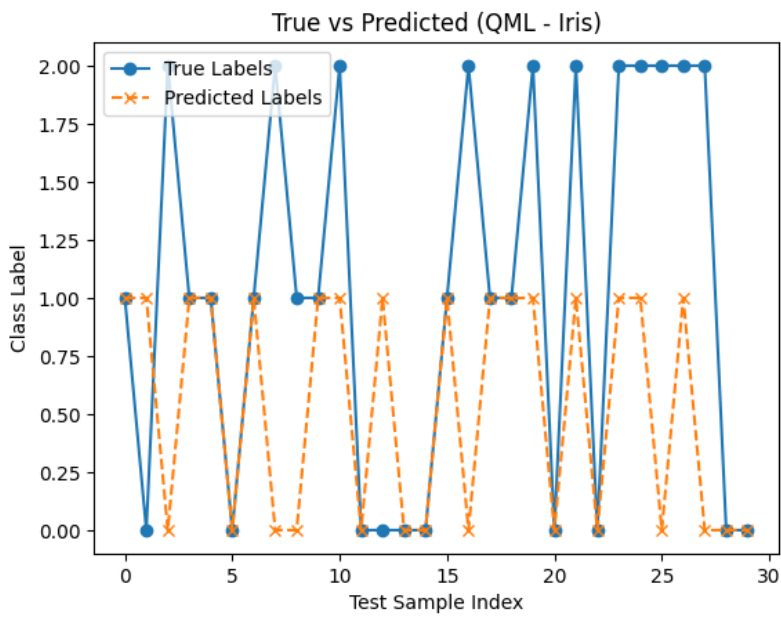
plt.plot(y_test, 'o-', label="True Labels")
plt.plot(y_pred, 'x--', label="Predicted Labels")

plt.xlabel("Test Sample Index")

```

```
plt.ylabel("Class Label")
plt.title("True vs Predicted (QML - Iris)")

plt.legend()
plt.show()
```



```
In [78]: errors = y_test != y_pred

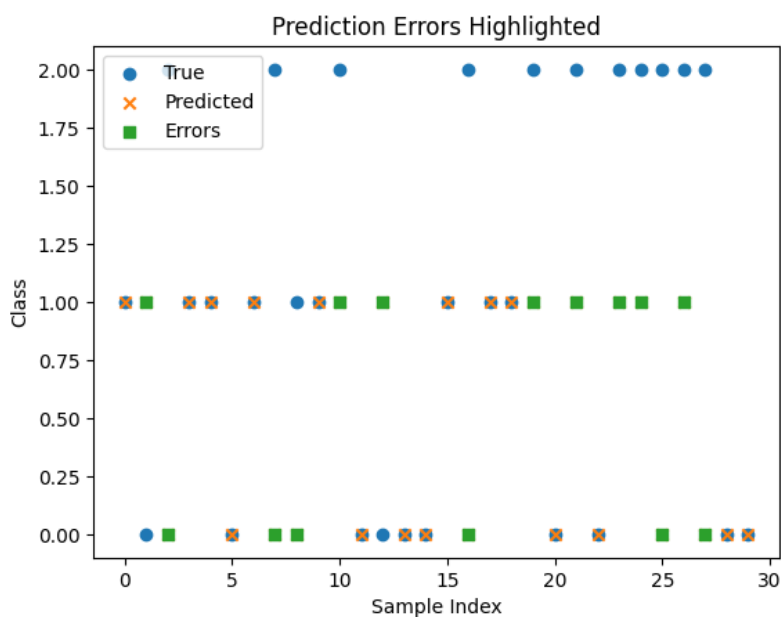
plt.figure()

plt.scatter(range(len(y_test)), y_test, label="True", marker='o')
plt.scatter(range(len(y_pred)), y_pred, label="Predicted", marker='x')

# Highlight wrong predictions
plt.scatter(
    np.where(errors),
    y_pred[errors],
    label="Errors",
    marker='s'
)

plt.xlabel("Sample Index")
plt.ylabel("Class")
plt.title("Prediction Errors Highlighted")

plt.legend()
plt.show()
```



```
In [79]: # Sort by true labels
sorted_idx = np.argsort(y_test)

y_test_sorted = y_test[sorted_idx]
```

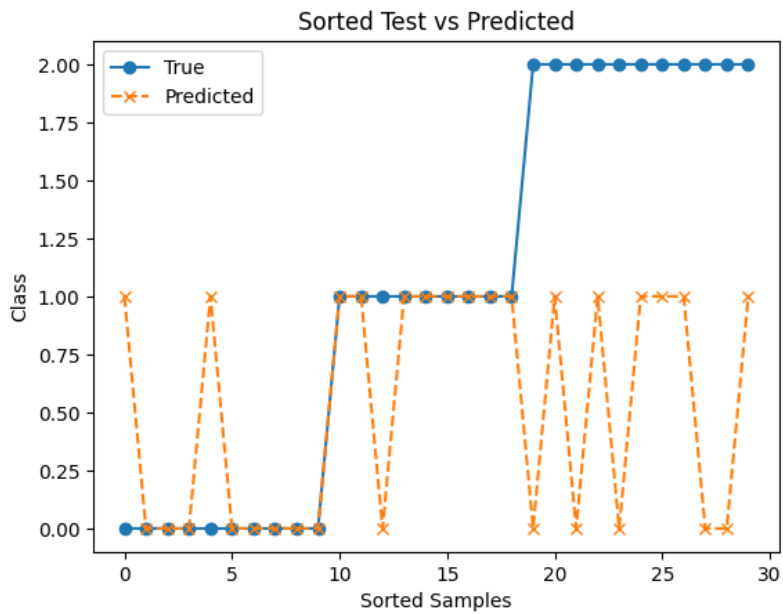
```
y_pred_sorted = y_pred[sorted_idx]

plt.figure()

plt.plot(y_test_sorted, 'o-', label="True")
plt.plot(y_pred_sorted, 'x--', label="Predicted")

plt.xlabel("Sorted Samples")
plt.ylabel("Class")
plt.title("Sorted Test vs Predicted")

plt.legend()
plt.show()
```



In []: